



Pareto optimization of two-agent scheduling on parallel batch machines

Cui-Lin Zhang ^a, Guo-Qiang Fan ^{b,*}

^a School of Economics and Management, Xihang University, 259 West Second Ring Road, Xi'an, 710077, Shaanxi, PR China

^b Department of Management Science & Engineering, Xidian University, 266 Xinglong Section of Xifeng Road, Xi'an, 710126, Shaanxi, PR China

ARTICLE INFO

Keywords:

Two-agent scheduling
Heuristics
Pareto optimization
Parallel batch machine
Makespan

ABSTRACT

This paper considers a Pareto optimization problem of scheduling jobs of two competing agents on parallel batch machines. The jobs have equal processing times and non-identical job sizes. The objective is to find Pareto optimality and the corresponding schedules for minimizing both agents' makespans. We analyze and identify an approximate Pareto region with a guarantee of 2-approximate Pareto optimal. We propose an integrated algorithm to find the approximate Pareto optimal points. Our computational study shows that the proposed algorithm outperforms the widely used non-dominated sorting genetic algorithm (NSGA-II), and that the obtained approximate Pareto optimal front is very close to the Pareto optimal front.

1. Introduction

This work is motivated by the assignment of different orders in a heat treatment workshop. Heat treating furnace heats and cools solid metal or alloy parts in a controlled manner to manipulate the material's microstructure, and further alter their mechanical properties such as hardness, strength, toughness, ductility, and elasticity. The management of heat treating furnace requires efforts. On the one hand, a heat treating furnace, as a parallel batch machine, can process several parts simultaneously as a batch, thereby improving throughput and balancing material flow in the production system. On the other hand, parallel batch machine is often a structural bottleneck in a production system. The number of parallel batch machines in a production system is usually no greater than a certain number due to factors such as equipment cost, energy consumption, installation space limitation, and environmental requirement. Naturally, operations on parallel batch machines have longer processing time compared with that on single-item machines. In practice, it often requires several hours for a batch operation while only several minutes for a non-batch operation. Overall, scheduling of jobs on parallel batch machines greatly influences the overall performance of a system, compared with that on standard machines processing a single job at a time (Wang et al., 2016, 2017).

Two kinds of orders, *original* and *new*, are considered in this work. Each order contains a set of parts with equal processing times and non-identical part sizes. The original order has been planned for processing on a parallel batch machine, but the processing has not yet begun. Then the new order arrives and competes for the use of the machine. We introduce *agents* for orders, and each order consists of a set of jobs. Each

agent prefers his order to be completed as soon as possible. Though jobs from the two agents can be treated without differentiation in scheduling decision, the resulted schedule may not be satisfactory to either agent. The production manager targets to find a schedule that balances the objectives of the two agents.

The above practical example motivates us to scheduling jobs from two competing agents on parallel batch machines. Each agent has a set of jobs to be processed. The objective of each agent is to minimize makespan, which is the maximum completion time of his order. The jobs of these two agents have equal processing times and non-identical sizes. Jobs have different sizes no greater than machine capacity. The parallel batch machine can process multiple jobs simultaneously as a batch as long as the total size of jobs in the batch does not exceed the machine capacity. Once started, a batch processing cannot be interrupted or changed until the processing is finished. The goal of this paper is to find *Pareto optimal points* and their corresponding schedules, which helps the production manager balance the makespans of the two agents. Specifically, in a *Pareto optimal schedule*, neither agent can reduce its makespan without increasing the makespan of the other agent, and a Pareto optimal point is a pair of makespans of a Pareto optimal schedule.

This study has made several contributions. First, we analyze the Pareto optimal points, and identify an approximate Pareto region such that any approximate Pareto point in this region is 2-approximate Pareto optimal. Second, we design an integrated algorithm under a unified framework in which the heuristics can be updated, encapsulated and added. The integrated algorithm combines four different heuristics together, unites all the obtained approximate Pareto optimal sets, and

* Corresponding author.

E-mail addresses: zhanglei@xaau.edu.cn (C.-L. Zhang), gqfan@xidian.edu.cn (G.-Q. Fan).

<https://doi.org/10.1016/j.eswa.2025.129637>

Received 10 April 2025; Received in revised form 16 August 2025; Accepted 3 September 2025

Available online 22 September 2025

0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

generates an approximate Pareto optimal set with high quality. Third, the integrated algorithm outperforms the widely used non-dominated sorting genetic algorithm (*NSGA-II*), and obtain a set of approximate Pareto optimal points which is very close to the set of Pareto optimal points.

The rest of the paper is organized as follows. In Section 2, we review the literature related to two-agent scheduling on parallel batch machines. In Section 3, the problem is described and modeled. Section 4 shows some theoretical results regarding the Pareto optimal points. We design an algorithm to find approximate Pareto optimal points in Section 5, and evaluate it computationally in Section 6. Finally, conclusions are presented in Section 7.

2. Literature review

The two-agent scheduling problem has attracted a large amount of attentions from the operations and manufacturing research communities. Baker and Smith (2003) and Agnetis et al. (2004) are the first to study the two-agent scheduling problem. Readers are referred to Perez-Gonzalez and Framinan (2014) and Agnetis et al. (2014) for surveys on two-agent scheduling.

For relevance, we review related literature of the two-agent scheduling on parallel batch machines, and introduce three widely considered models: linear combination optimization, ϵ -constraint approach, and Pareto optimization.

In the linear combination optimization, the goal is to minimize a linear combination of two agents' objective functions. Sabouni and Jolai (2010) consider a two-agent scheduling on a single parallel batch machine to minimize a linear function of the makespan of agent A and the maximum lateness of agent B . Specially, following the three-field notation of Graham et al. (1979), the problems are denoted by $1|p\text{-batch}, b < n|\alpha C_{\max}^A + (1 - \alpha)L_{\max}^B$ and $1|p\text{-batch}, b = \infty|\alpha C_{\max}^A + (1 - \alpha)L_{\max}^B$. Here, $p\text{-batch}$ denotes parallel batch machine, n is the number of jobs, and b is the maximum number of jobs in a batch. Constraints $b < n$ and $b = \infty$ mean parallel batch machines are bounded and unbounded, respectively. Parameter $0 \leq \alpha \leq 1$ is a weight factor. Objectives $C_{\max}^A$ and $L_{\max}^B$ mean that the makespan of agent A and the maximum lateness of agent B are to be minimized. Sabouni and Jolai (2010) propose a dynamic programming algorithm for the case with unbounded machine capacity and incompatible jobs, which means jobs from different agents cannot be grouped into the same batch. Feng et al. (2013) show that the above algorithm is suboptimal, and develop a polynomial time optimal algorithm. For cases with bounded machine, Sabouni and Jolai (2010) also design two dynamic programming algorithms for incompatible and compatible jobs, respectively. Zhang et al. (2019) study a two-agent scheduling problem with unit job processing time and non-identical job sizes to minimize a linear combination of the makespans of two agents, denoted by $1|p\text{-batch}, p_j^X = 1, s_j^X \leq C|\alpha C_{\max}^A + (1 - \alpha)C_{\max}^B$, where $p_j^X = 1$ means each job of agent X has unit processing time, $s_j^X \leq C$ means each job size of agent X is not greater than the machine capacity C , for $X \in \{A, B\}$. These authors design an approximation algorithm, and analyse its absolute worst case performance ratio and its asymptotic worst case performance ratio, both of which are linear functions of the weight factor α . Fan et al. (2023) study a two-agent scheduling problem on mixed batch machines to minimize a linear combination of two agents' makespans, design four approximation algorithms and analyze their worst-case performance ratios. Li et al. (2022) study a problem on uniform parallel batch machines to minimize the maximum lateness and the total pollution emission costs, and design a discrete bi-objective evolutionary algorithm.

In the ϵ -constraint approach, the goal is to minimize the objective of one agent subject to an upper bound on the objective of the other agent. Li and Yuan (2012) consider a two-agent scheduling problem on an unbounded parallel batch machine, denoted by $1|p\text{-batch}, b = \infty, f^B \leq \epsilon|f^A$, where ϵ is a given constant. Costs f^A and f^B are regular objective functions of agents A and B , respectively. These authors propose

polynomial and pseudopolynomial time algorithms for various combinations of regular objective functions and compatibility categories of jobs. Further, Fan et al. (2013) consider a two-agent scheduling problem on a bounded parallel batch machine to minimize the makespan of agent A , subjected to a restriction on the agent B 's makespan, i.e., problem $1|p\text{-batch}, b < n, C_{\max}^B \leq \epsilon|C_{\max}^A$. The authors show that the problem is polynomially solvable if jobs are incompatible, and weakly NP -hard if jobs are compatible. The authors also consider a two-agent scheduling problem to minimize the total completion time of agent A , subject to a restriction on agent B 's total completion time, i.e., problem $1|p\text{-batch}, b < n, C_{\max}^B \leq \epsilon|\sum C_j^A$. They prove that the problem is NP -hard, and polynomially solvable if jobs are incompatible and with a fixed number of job types. Wang et al. (2017) study a two-agent scheduling problem with non-identical job sizes, denoted by $1|p\text{-batch}, p_j^X = 1, s_j^X \leq C, C_{\max}^B \leq \epsilon|C_{\max}^A$. They prove that there is no polynomial-time algorithm with a finite worst-case performance ratio, unless $P = NP$. Furthermore, these authors employ two strategies for meeting constraint $C_{\max}^B \leq \epsilon$, and correspondingly develop two efficient approximate algorithms to solve the problem.

In the Pareto optimization, the goal is to find all the Pareto optimal points, denoted by $\mathcal{P}(\cdot, \cdot)$. He et al. (2007) study a two-agent scheduling problem on an unbounded parallel batch machine, denoted by $1|p\text{-batch}, b = \infty|\mathcal{P}(C_{\max}^A, L_{\max}^B)$. They design a polynomial time algorithm to find all the Pareto optimal points and recognize an upper bound on the number of Pareto optimal points. Sabouni and Jolai (2010) study a two-agent scheduling problem on a bounded parallel batch machine, denoted by $1|p\text{-batch}, b \leq n, p_j^B = 1|\mathcal{P}(C_{\max}^A, L_{\max}^B)$. They develop an optimal algorithm to obtain all the Pareto optimal points. Tan et al. (2011) study a two-agent schedule problem with non-identical sizes on a single parallel batch machine, denoted by $1|p\text{-batch}, s_j^X \leq C|\mathcal{P}(C_{\max}^A, C_{\max}^B)$, and develop an ant colony algorithm to find Pareto optimal points. He et al. (2022) study a Pareto optimization problem on parallel batch machines, where the objective of agent A is maximum cost and that of agent B is makespan, and design a polynomial time algorithm to find all the Pareto optimal schedules.

The problem investigated in this work is a special case of the problem studied by Tan et al. (2011). Specifically, they allow arbitrary job processing times, while we assume unit job processing time. Tan et al. (2011) develop a meta-heuristic to search for Pareto optimal points. Our focus is to identify theoretically where all the Pareto optimal points are located, and to find approximate Pareto optimal points with provable guarantees. That is, the results by Tan et al. (2011) are computational, while our results are more theoretical.

3. Problem description and modeling

We consider parallel batch machines that can process a number of jobs simultaneously as a batch. The machine has bounded capacity. The jobs have non-identical sizes. The total size of jobs in the batch does not exceed machine capacity. The jobs have unit job processing time, i.e., $p_j^X = 1, j = 1, 2, \dots, n_X, X \in \{A, B\}$. For a parallel batch, the batch processing time is equal to the maximum processing time of jobs in the batch. Hence, all the batches have unit processing time. Once started, batch processing cannot be interrupted or changed, i.e., preemption is not allowed. A job cannot be allocated to more than one batch, splitting is not allowed. All jobs are available for processing at time zero (Table 1).

Two agents, denoted by A and B , have n_A and n_B jobs to be processed on batch machines, respectively. Job sets of agents A and B are not empty, i.e., $\mathcal{J}^A = \{J_1^A, J_2^A, \dots, J_{n_A}^A\} \neq \emptyset$ and $\mathcal{J}^B = \{J_1^B, J_2^B, \dots, J_{n_B}^B\} \neq \emptyset$. Let $\mathcal{J}^{AB} = \mathcal{J}^A \cup \mathcal{J}^B$ denote all jobs provided that $\mathcal{J}^A$ -jobs and $\mathcal{J}^B$ -jobs are treated without differentiation. For preciseness, we use terms $\mathcal{J}^A$ -jobs, $\mathcal{J}^B$ -jobs and $\mathcal{J}^{AB}$ -jobs to refer to jobs from $\mathcal{J}^A$, $\mathcal{J}^B$ and $\mathcal{J}^{AB}$, respectively. The two agents have no job in common, and thus $\mathcal{J}^A \cap \mathcal{J}^B = \emptyset$. Jobs from different agents can be grouped into one batch, i.e., jobs are compatible.

Table 1
Notations for problem.

Notation	Meaning
C	Machine capacity
J^X	Job set of agent X , $X \in \{A, B\}$
n_X	Number of jobs of agent X , $X \in \{A, B\}$
n	Total number of jobs, i.e., $n = n_A + n_B$
J_j^X	j -th job of agent X , $X \in \{A, B\}$
p_j^X	Processing time of job J_j^X
s_j^X	Size of job J_j^X
$C_{\max}^X$	Makespan of jobs from agent X

Table 2
Notations of Pareto optimal region.

Notation	Meaning
R_{PF}	A region covering the Pareto front
R_{PF}^1, R_{PF}^2	Two small regions, and $R_{PF} = R_{PF}^1 \cup R_{PF}^2$
R_{approx}	Approximation region covering the Pareto front
N_{PF}^1	Number of Pareto optimal points in region R_{PF}^1
N_{PF}^2	Number of Pareto optimal points in region R_{PF}^2
N_{PF}^+	Number of Pareto optimal points in region R_{PF}
$C_{OPT}^X, X \in \{A, B, AB\}$	Makespan of an optimal schedule for J^X -jobs
$C_{FFD}^X, X \in \{A, B, AB\}$	Makespan of J^X -jobs obtained by FFD rule
C_{FFD}^+	$C_{FFD}^+ = C_{FFD}^A + C_{FFD}^B$
$C_{LB}^X, X \in \{A, B, AB\}$	Lower bound on the makespan for J^X -jobs
S_{LB}	Lower bound set of Pareto front
S_{RF}	Reference set of Pareto front

Agents A and B prefer to minimize the makespans of their own jobs. The goal of our problem is to find Pareto optimal points and their corresponding schedules. A Pareto optimal schedule is a schedule in which neither agent can reduce his makespan without increasing the makespan of the other agent. A Pareto optimal schedule specifies a unique pair of makespans of jobs from the two agents, which is a Pareto optimal point. Note that a Pareto optimal point may correspond to multiple Pareto optimal schedules. The set of all Pareto optimal schedules is the *Pareto optimal set*. The set of all Pareto optimal points is the *Pareto front*. Following the three-field notation of [Graham et al. \(1979\)](#), we denote our problem as $Pm|p\text{-batch}, p_j^X = 1, s_j^X \leq C|\mathcal{P}(C_{\max}^A, C_{\max}^B)$, where $\mathcal{P}(C_{\max}^A, C_{\max}^B)$ denotes the Pareto front.

Considering the problem that agent A has only one job with size C , the job from agent A occupies a full batch. The problem is equivalent to minimize the number of batches used by agent B , which is a bin packing problem. According to [Garey and Johnson \(1979\)](#), bin packing problem is strongly NP -hard. Thus, our problem is strongly NP -hard, and it is impossible to obtain a Pareto optimal schedule in polynomial time, unless $P = NP$.

Hence, we develop algorithms to find *approximate Pareto optimal schedules*. Specifically, given a constant ρ , a $(1 + \rho)$ -approximate Pareto optimal schedule σ is a feasible schedule such that there is no other feasible schedule σ' satisfying $(1 + \rho)C_{\max}^X(\sigma') \leq C_{\max}^X(\sigma)$ for $X \in \{A, B\}$, with at least one strict inequality ([Agnietis et al., 2014](#); [Ehrgott et al., 2011](#)). An approximate Pareto optimal schedule specifies a pair of makespans of jobs from the two agents, which is an *approximate Pareto optimal point*.

4. Approximation of Pareto front

This section identifies some properties of the Pareto front, including a region covering the Pareto front, an *approximate Pareto region*, and a lower bound set and a reference set to evaluate approximate Pareto optimal points. For convenience, we give some notations of this section in [Table 2](#).

4.1. Pareto optimal region analysis

We first recognize four Pareto optimal points, P_1, P_2, P_3, P_4 , and then use these points to determine a region covering the Pareto front, as il-

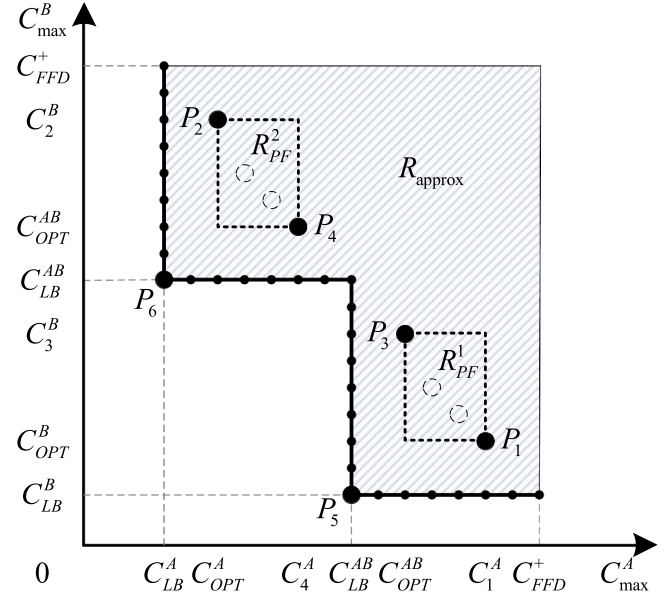


Fig. 1. Pareto front and approximate Pareto region.

lustrated in [Fig. 1](#). On the x and y axes, we report the makespans of J^A -jobs and J^B -jobs, respectively.

Let C_{OPT}^A (respectively, C_{OPT}^B) be the makespan of an optimal schedule when only J^A -jobs (respectively, J^B -jobs) are considered. C_{OPT}^{AB} denote the makespan of an optimal schedule when only J^{AB} -jobs are considered. Points P_1, P_2, P_3, P_4 are defined as follows.

- Point P_1 is located at (C_1^A, C_{OPT}^B) , where C_1^A is the minimum makespan of J^A -jobs of all the schedules that the makespan of J^B -jobs is C_{OPT}^B .
- Point P_2 is located at (C_{OPT}^A, C_2^B) , where C_2^B is the minimum makespan of J^B -jobs of all the schedules that the makespan of J^A -jobs is C_{OPT}^A .
- Point P_3 is located at (C_{OPT}^{AB}, C_3^B) , where C_3^B is the minimum makespan of J^B -jobs of all the schedules that the makespan of J^A -jobs is C_{OPT}^{AB} .
- Point P_4 is located at (C_4^A, C_{OPT}^{AB}) , where C_4^A is the minimum makespan of J^A -jobs of all the schedules that the makespan of J^B -jobs is C_{OPT}^{AB} .

Obviously, all the four points P_1, P_2, P_3 and P_4 are Pareto optimal points. Using the four points, we define two regions as follows.

$$R_1 = \{(u^A, v^B) : C_{OPT}^A \leq u^A \leq C_1^A, C_{OPT}^B \leq v^B \leq C_2^B\},$$

$$R_2 = \{(u^A, v^B) : u^A \geq C_{OPT}^{AB}, v^B \leq C_3^B\} \cup \{(u^A, v^B) : u^A \leq C_4^A, v^B \geq C_{OPT}^{AB}\}.$$

Region $R_{PF} = R_1 \cap R_2$ is bordered by dark dotted lines in [Fig. 1](#). For region R_{PF} , we have the following theorem.

Theorem 1. Region R_{PF} covers the Pareto front.

Proof. Since C_{OPT}^A and C_{OPT}^B are the minimum makespans of J^A -jobs and J^B -jobs respectively, all the feasible points and hence all the Pareto optimal points are in region $\{(u^A, v^B) : u^A \geq C_{OPT}^A, v^B \geq C_{OPT}^B\}$. Feasible points in region $\{(u^A, v^B) : u^A > C_1^A, v^B \geq C_{OPT}^B\}$ are dominated by point $P_1(C_1^A, C_{OPT}^B)$, thus all the Pareto optimal points are in region $\{(u^A, v^B) : C_{OPT}^A \leq u^A \leq C_1^A, v^B \geq C_{OPT}^B\}$. All the feasible points in region $\{(u^A, v^B) : C_{OPT}^A \leq u^A \leq C_1^A, v^B > C_2^B\}$ are dominated by point $P_2(C_{OPT}^A, C_2^B)$, thus all the Pareto optimal points are in the region between points P_1 and P_2 , i.e., region R_1 .

Since C_{OPT}^{AB} is the minimum makespan of all jobs, for a feasible schedule, at least one of the makespans of J^A -jobs and J^B -jobs is not less than C_{OPT}^{AB} . Thus, there is not any feasible point in the region between

points P_3 and P_4 , i.e., $\{(u^A, v^B) : u^A < C_{OPT}^{AB}, v^B < C_{OPT}^{AB}\}$. The feasible points in regions $\{(u^A, v^B) : u^A > C_{OPT}^{AB}, v^B > C_3^B\}$ and $\{(u^A, v^B) : u^A > C_4^A, v^B > C_{OPT}^{AB}\}$ are dominated by points P_3 and P_4 , respectively. Thus, all the Pareto optimal points are in region R_2 .

Therefore, $R_{PF} = R_1 \cap R_2$ covers all the Pareto optimal points. $\square$

Note that region R_{PF} consists of two separate regions. We next discuss the total number of Pareto optimal points in the two regions. Let R_{PF}^1 be the region between points P_1 and P_3 , and R_{PF}^2 be the region between points P_2 and P_4 . Thus, $R_{PF} = R_{PF}^1 \cup R_{PF}^2$. Let N_{PF}^1 and N_{PF}^2 be the numbers of Pareto optimal points in regions R_{PF}^1 and R_{PF}^2 , respectively. For upper bounds on N_{PF}^1 and N_{PF}^2 , we have the following theorem.

Theorem 2. *The number of Pareto optimal points in regions R_{PF}^1 is not greater than $\min\{C_3^B - C_{OPT}^B + 1, C_1^A - C_{OPT}^{AB} + 1\}$, and in region R_{PF}^2 , it is not greater than $\min\{C_4^A - C_{OPT}^A + 1, C_2^B - C_{OPT}^{AB} + 1\}$.*

Proof. By contradiction. Assume $N_{PF}^1 > \min\{C_3^B - C_{OPT}^B + 1, C_1^A - C_{OPT}^{AB} + 1\}$. Let S_{PF}^1 be the set of all the Pareto optimal points in region R_{PF}^1 . Without loss of generality, we assume $C_3^B - C_{OPT}^B + 1 \leq C_1^A - C_{OPT}^{AB} + 1$, and thus $N_{PF}^1 = |S_{PF}^1| > C_3^B - C_{OPT}^B + 1$. Note that the Pareto optimal points are all integer points. There are $C_3^B - C_{OPT}^B + 1$ different integer values of x -axis in set S_{PF}^1 . Thus, $N_{PF}^1 = |S_{PF}^1| > C_3^B - C_{OPT}^B + 1$ means there are two different integer points in set S_{PF}^1 with the same value of x -axis. One of these two points dominates another point, which is a contradiction that S_{PF}^1 is a set of Pareto optimal points.

Similarly, we have $N_{PF}^2 \leq \min\{C_4^A - C_{OPT}^A + 1, C_2^B - C_{OPT}^{AB} + 1\}$. $\square$

Let N_{PF}^* be the total number of Pareto optimal points. According to Theorem 1, $N_{PF}^* \leq N_{PF}^1 + N_{PF}^2 \leq \min\{C_4^A - C_{OPT}^A + 1, C_2^B - C_{OPT}^{AB} + 1\} + \min\{C_3^B - C_{OPT}^B + 1, C_1^A - C_{OPT}^{AB} + 1\}$.

4.2. Approximate Pareto region analysis

Due to the NP-hardness of our problem, it is impossible to obtain C_{OPT}^A , C_{OPT}^B and C_{OPT}^{AB} in polynomial time, unless $P = NP$. Consequently, we have no alternative but to settle for the second best by identifying an *approximate Pareto region*, which covers region R_{PF} and hence the Pareto front, and is obtainable in polynomial time.

In order to locate the region, we analyze upper and lower bounds on the minimum makespan of each agent. Let C_{FFD}^X be the makespan of J^X -jobs ($X \in \{A, B, AB\}$) in a schedule formed following the first fit decreasing (FFD) rule (Johnson, 1973). Specifically, the FFD rule first arranges jobs in a nonincreasing order of their sizes, and then in this order assigns one job at a time to the first batch with enough space. The batches obtained by FFD rule is assigned to machines equally. Let N_{FFD}^X be the number of batches obtained by FFD rule, and $C_{FFD}^X = \lceil N_{FFD}^X / m \rceil$. Let $C_{FFD}^+ = C_{FFD}^A + C_{FFD}^B$, thus C_{FFD}^+ is an upper bound on the minimum makespan of each agent.

Let C_{LB}^X be a lower bound on the minimum makespan for J^X -jobs, $X \in \{A, B, AB\}$. To determine the three lower bounds C_{LB}^A , C_{LB}^B and C_{LB}^{AB} , we introduce two lower bound functions established by Dósa et al. (2013) and Martello and Toth (1990), denoted by $f_D(J^X)$ and $f_M(J^X)$ respectively.

$$f_D(J^X) = \left\lceil \frac{9N_{FFD}^X - 6}{11m} \right\rceil,$$

$$f_M(J^X) = \left\lceil \frac{1}{m} \cdot \max_{K \in [0, C/2] \cap \mathbb{Z}} \{ |N_1^X| + \max\{|N_2^X|, \lceil \frac{1}{C} \sum_{J_j^X \in N_2^X \cup N_3^X} s_j^X \} \} \right\rceil,$$

where $X \in \{A, B, AB\}$, $N_1^X = \{J_j^X \in J^X : s_j^X > C - K\}$, $N_2^X = \{J_j^X \in J^X : C/2 < s_j^X \leq C - K\}$ and $N_3^X = \{J_j^X \in J^X : K \leq s_j^X \leq C/2\}$.

We consider the maximum of these two lower bound functions, i.e., $C_{LB}^X = \max\{f_D(J^X), f_M(J^X)\}$, $X \in \{A, B, AB\}$. Further, using these

lower bounds, we define the following two regions R_3 and R_4 .

$$R_3 = \{(u^A, v^B) : C_{LB}^A \leq u^A \leq C_{FFD}^+, C_{LB}^B \leq v^B \leq C_{FFD}^+\},$$

$$R_4 = \{(u^A, v^B) : u^A \geq C_{LB}^{AB}\} \cup \{(u^A, v^B) : v^B \geq C_{LB}^{AB}\}.$$

Region $R_{approx} = R_3 \cap R_4$ is an approximate Pareto region covering region R_{PF} and hence the Pareto front, as illustrated as the shaded area in Fig. 1.

Theorem 3. *Region R_{approx} covers the Pareto front.*

Proof. We prove this theorem by showing $R_{approx} \supseteq R_{PF}$, where region R_{PF} covers the Pareto front according to Theorem 1.

Note that $R_4 = \{(u^A, v^B) : u^A \geq C_{LB}^{AB}\} \cup \{(u^A, v^B) : v^B \geq C_{LB}^{AB}\}$, and $R_3 = \{(u^A, v^B) : u^A \geq C_{OPT}^{AB}, v^B \leq C_3^B\} \cup \{(u^A, v^B) : u^A \leq C_4^A, v^B \geq C_{OPT}^{AB}\}$. Since $C_{LB}^{AB} \leq C_{OPT}^{AB}$, we obtain $R_4 \supseteq R_2$.

We consider a feasible schedule in which an optimal schedule of J^B -jobs is sequenced first and followed by an optimal schedule of J^A -jobs. In this schedule, the makespan of J^B -jobs is C_{OPT}^B and the makespan of J^A -jobs is $C_{OPT}^A + C_{OPT}^B$. Since C_1^A is the minimum makespan of J^A -jobs subject to the makespan of J^B -jobs is C_{OPT}^B , we have $C_1^A \leq C_{OPT}^A + C_{OPT}^B \leq C_{FFD}^A + C_{FFD}^B = C_{FFD}^+$. Similarly, we have $C_2^B \leq C_{FFD}^+$. According to the definition of C_{OPT}^A and C_{OPT}^B , we have $C_{LB}^A \leq C_{OPT}^A$ and $C_{LB}^B \leq C_{OPT}^B$. Note that $R_3 = \{(u^A, v^B) : C_{LB}^A \leq u^A \leq C_{FFD}^+, C_{LB}^B \leq v^B \leq C_{FFD}^+\}$ and $R_1 = \{(u^A, v^B) : C_{OPT}^A \leq u^A \leq C_1^A, C_{OPT}^B \leq v^B \leq C_2^B\}$, thus we have $R_3 \supseteq R_1$.

Therefore, we have $R_{approx} = R_3 \cap R_4 \supseteq R_1 \cap R_2 = R_{PF}$. $\square$

For approximation of the Pareto front, the following theorem provides a guarantee of the quality of approximate Pareto optimal points found in region R_{approx} .

Theorem 4. *Any approximate Pareto optimal point found in region R_{approx} is 2-approximate Pareto optimal.*

Proof. Since C_{OPT}^{AB} is the minimum makespan of all jobs, for a feasible point (u^A, v^B) , at least one of the makespans of J^A -jobs and J^B -jobs is not less than C_{OPT}^{AB} , i.e., at least one of the inequations $u^A \geq C_{OPT}^{AB}$ and $v^B \geq C_{OPT}^{AB}$ holds. For any approximate Pareto optimal point (r^A, s^B) found in region R_{approx} , we have $r^A \leq C_{FFD}^+$ and $s^B \leq C_{FFD}^+$.

In the schedule that J^X -jobs are processed using FFD rule, the total size of jobs in any two batches is not less than machine capacity C , thus $mC_{FFD}^X \leq 2 \sum_{j=1}^{n_X} s_j^X / C$, $X \in \{A, B\}$. Obviously, $mC_{OPT}^{AB} \geq (\sum_{j=1}^{n_A} s_j^A + \sum_{j=1}^{n_B} s_j^B) / C$. Thus, $mC_{FFD}^+ = mC_{FFD}^A + mC_{FFD}^B \leq 2(\sum_{j=1}^{n_A} s_j^A + \sum_{j=1}^{n_B} s_j^B) / C \leq 2mC_{OPT}^{AB}$.

Thus, at least one of the inequations $r^A \leq C_{FFD}^+ \leq 2C_{OPT}^{AB} \leq 2u^A$ and $s^B \leq C_{FFD}^+ \leq 2C_{OPT}^{AB} \leq 2v^B$ holds, i.e., there is no point satisfying the two inequations $2u^A \leq r^A$ and $2v^A \leq s^A$ with at least one strict inequality. Therefore, according to the definition, point (r^A, s^B) is 2-approximate Pareto optimal. $\square$

Furthermore, by applying lower bound function $f_D(J^X)$, we establish the following upper bound on the number of Pareto optimal points.

Theorem 5. *The total number of Pareto optimal points is not greater than $\frac{17}{11} + \frac{2}{11}(C_{FFD}^A + C_{FFD}^B) + \frac{9}{11} \min\{C_{FFD}^A, C_{FFD}^B\}$.*

Proof. Since $R_3 \supseteq R_{approx}$, the total number of Pareto optimal points is not greater than that in region R_3 . Similar to the discussion in Theorem 2, the number of Pareto optimal points in region R_3 is not greater than $\min\{C_{FFD}^+ - C_{LB}^A + 1, C_{FFD}^+ - C_{LB}^B + 1\}$. By applying lower bound function $f_D(J^X)$, we have $C_{LB}^X \geq (9N_{FFD}^X - 6) / 11m$, $X \in \{A, B\}$. Thus, we have $\min\{C_{FFD}^+ - C_{LB}^A + 1, C_{FFD}^+ - C_{LB}^B + 1\} \leq 3 + 6/11m + 2(N_{FFD}^A + N_{FFD}^B) / 11 + 9 \min\{N_{FFD}^A, N_{FFD}^B\} / 11$. $\square$

4.3. Lower bound set and reference set

For a single-objective optimization problem, a common method to evaluate the performance of an algorithm is to compare the obtained

objective value with a lower bound on the optimal objective value. Following this idea, for our two-objective optimization problem, we examine lower bound of the Pareto front. The lower boundary of the approximate Pareto region R_{approx} is the stairwise line in Fig. 1, which can be interpreted as a lower bound on the Pareto front. Note that all the points in the Pareto front are integer points. The set of integer points on the lower boundary is called the *lower bound set*, denoted as S_{LB} . Specifically,

$$S_{LB} = \{(u^A, v^B) : u^A = C_{LB}^A, C_{LB}^{AB} \leq v^B \leq C_{FFD}^+, u^A, v^B \in \mathbb{Z}\} \cup \\ \{(u^A, v^B) : C_{LB}^A \leq u^A \leq C_{LB}^{AB}, v^B = C_{LB}^{AB}, u^A, v^B \in \mathbb{Z}\} \cup \\ \{(u^A, v^B) : u^A = C_{LB}^{AB}, C_{LB}^B \leq v^B \leq C_{LB}^{AB}, u^A, v^B \in \mathbb{Z}\} \cup \\ \{(u^A, v^B) : C_{LB}^{AB} \leq u^A \leq C_{FFD}^+, v^B = C_{LB}^B, u^A, v^B \in \mathbb{Z}\}$$

where $\mathbb{Z}$ is the integer set.

We investigate two important points, $P_5(C_{LB}^{AB}, C_{LB}^B)$ and $P_6(C_{LB}^A, C_{LB}^{AB})$ in the lower bound set S_{LB} , as shown in Fig. 1.

Theorem 6. *If P_5 and P_6 are both obtainable by feasible schedules, then set $\{P_5, P_6\}$ is the Pareto front.*

Proof. Note that points P_5 and P_6 are located at (C_{LB}^{AB}, C_{LB}^B) and (C_{LB}^A, C_{LB}^{AB}) , respectively, and that $C_{LB}^X \leq C_{OPT}^X$, $X \in \{A, B, AB\}$. Since P_5 and P_6 are both obtainable by feasible schedules, we have $C_{LB}^{AB} = C_{OPT}^{AB}$, $C_{LB}^B = C_{OPT}^B$, and $C_{LB}^A = C_{OPT}^A$. Thus points P_5 and P_6 are located at $(C_{OPT}^{AB}, C_{OPT}^B)$ and $(C_{OPT}^A, C_{OPT}^{AB})$, respectively, which means points P_1 , P_3 and P_5 are the same point, and points P_2 , P_4 and P_6 are the same point. Each of regions R_{PF}^1 and R_{PF}^2 contains only one point, i.e., $R_{PF}^1 = \{P_5(C_{OPT}^{AB}, C_{OPT}^B)\}$ and $R_{PF}^2 = \{P_6(C_{OPT}^A, C_{OPT}^{AB})\}$. According to Theorem 1, the Pareto front are in region $R_{PF}^1 \cup R_{PF}^2 = R_{PF}$, and thus set $\{P_5, P_6\}$ is the Pareto front. $\square$

Theorem 6 indicates that, if we obtain a set of approximate Pareto optimal points containing P_5 and P_6 , then we obtain the Pareto front. We use the distance between a set of approximate Pareto optimal points and set $\{P_5, P_6\}$ to estimate the quality of the approximate Pareto optimal points. Thus, we call set $\{P_5, P_6\}$ a *reference set*, denoted by S_{RS} .

We introduce two distance metrics established by Czyzżak and Jaszkiewicz (1998) and Ulungu et al. (1998). D_{avg} (respectively, D_{max}) is the average (respectively, maximum) distance from a set of approximate Pareto points S_{approx} to reference set S_{RS} , and are defined as follows.

$$D_{avg} = \frac{\sum_{P' \in S_{RS}} \min_{P \in S_{approx}} d(P, P')}{|S_{RS}|},$$

$$D_{max} = \max_{P' \in S_{RS}} \left\{ \min_{P \in S_{approx}} d(P, P') \right\},$$

where P denoting (a, b) is a point in S_{approx} , P' denoting (a', b') is a point in S_{RS} , and $d(P, P') = \max\{(a - a')/\Delta_A, (b - b')/\Delta_B\}$, with Δ_A and Δ_B are the range of x -axis and y -axis in region R_{approx} respectively, i.e., $\Delta_A = C_{FFD}^+ - C_{LB}^A$, $\Delta_B = C_{FFD}^+ - C_{LB}^B$. Since $a \leq C_{FFD}^+$ and $b \leq C_{FFD}^+$, and at least one of the inequations $a \geq C_{LB}^A$ and $b \geq C_{LB}^B$ holds, we have $0 \leq d(P, P_5) \leq 1$ and $0 \leq d(P, P_6) \leq 1$. Thus, we have $0 \leq D_{avg} \leq 1$ and $0 \leq D_{max} \leq 1$. Following the idea of Wang et al. (2015), we have the following Theorem.

Theorem 7. *The obtained set of approximate Pareto optimal points is the reference set if and only if $D_{avg} = 0$ or $D_{max} = 0$, i.e., $S_{approx} = S_{RS} \Leftrightarrow D_{avg} = 0 \Leftrightarrow D_{max} = 0$.*

Proof. We prove this theorem by the following two steps.

(1) $S_{approx} = S_{RS} \Rightarrow D_{avg} = 0$, $S_{approx} = S_{RS} \Rightarrow D_{max} = 0$.
Since $d(P_5, P_6) = d(P_6, P_5) = 1$ and $d(P_5, P_5) = d(P_6, P_6) = 0$, we have $\min_{P \in S_{approx}} d(P, P_5) = 0$ and $\min_{P \in S_{approx}} d(P, P_6) = 0$. Thus, $D_{avg} = 0$ and $D_{max} = 0$.

(2) $D_{avg} = 0 \Rightarrow S_{approx} = S_{RS}$, $D_{max} = 0 \Rightarrow S_{approx} = S_{RS}$.
Since $0 \leq D_{avg} \leq 1$ and $0 \leq D_{max} \leq 1$, $D_{avg} = 0$ or $D_{max} = 0$ means $\min_{P \in S_{approx}} d(P, P_5) = 0$ and $\min_{P \in S_{approx}} d(P, P_6) = 0$.

We prove $S_{RS} = \{P_5, P_6\} \subseteq S_{approx}$ by contradiction. Without loss of generality, we assume $P_5 \notin S_{approx}$. For $\forall P_k \in S_{approx}$, we have either $v_k - v_5 > 0$ or $u_k - u_5 > 0$, thus $d(P_k, P_5) = \max\{(u_k - u_5)/\Delta_A, (v_k - v_5)/\Delta_B\} > 0$ and $\min_{P \in S_{approx}} d(P, P_5) > 0$. Thus, we have $D_{max} > 0$ and $D_{avg} > 0$, which is a contradiction.

With the same argument for point P_6 , we have $P_6 \in S_{approx}$, and $\{P_5, P_6\} \subseteq S_{approx}$. Because S_{approx} is an obtained set of approximate Pareto optimal points and any point P' ($P' \in R_{approx} \setminus \{P_5, P_6\}$) is dominated by point P_5 or P_6 , we have $S_{approx} = S_{RS} = \{P_5, P_6\}$. $\square$

5. Integrated algorithm

Since our problem is NP -hard, it is impossible to obtain a Pareto optimal set in polynomial time unless $P = NP$. We have no alternative but to seek an approximate Pareto optimal set. Consequently, the key point of our study is to design effective heuristics for our problem. However, each heuristic is effective for specifically featured instances. With more variety of heuristics combined, the obtained approximate Pareto optimal set improves. Therefore, it is better to design a variety of heuristics, combine the obtained sets from different heuristics together, and return an approximate Pareto optimal set with high quality.

We develop an *integrated algorithm* based on the ϵ -constraint method consisting of three components. (1) Problem Transformation: transform the Pareto optimization problem into single objective problems. (2) Concurrent Problem-solving: solve the single objective problems with different heuristics, and obtain different approximate Pareto optimal sets. (3) Domination Operation: unite all the obtained approximate Pareto optimal sets, delete the dominated points in the union set, and generate an approximate Pareto optimal set with high quality. The algorithm is illustrated in Fig. 2, where notations will be explained later.

The integrated algorithm offers the capability of combining different heuristics together. This algorithm is based on a unified framework and standard solving flow. First, our algorithm uses a standard procedure in the problem transformation component. Then, the algorithm uses a fixed algorithm interface to apply different heuristics for the transformed single objective problems in the concurrent problem-solving component. Second, the heuristic library for the transformed single objective problems can be updated, encapsulated and added. Third, our algorithm can be adapted to parallel computing environment with multiple nodes and multithreads that execute different heuristics with different scheduling rules in parallel.

5.1. Component (1): problem transformation

Problem transformation is based on the ϵ -constraint method. The ϵ -constraint method selects only one objective function as optimization objective, called primary objective, and transforms the other objective function into one or several constraints (Haimes, 1971). Using the ϵ -constraint method, the original problem $Pm|p\text{-batch}, p_j^X = 1, s_j^X \leq C|P(C_{max}^A, C_{max}^B)$ is transformed into two constrained optimization problems $Pm|p\text{-batch}, p_j^X = 1, s_j^X \leq C, C_{max}^B \leq \epsilon|C_{max}^A$ and $Pm|p\text{-batch}, p_j^X = 1, s_j^X \leq C, C_{max}^A \leq \epsilon|C_{max}^B$, short for $C_{max}^A : C_{max}^B \leq \epsilon$ and $C_{max}^B : C_{max}^A \leq \epsilon$, respectively.

According to Theorem 7 in Hoogeveen (2005), there is an optimal schedule for constrained optimization problem so that this optimal schedule is also Pareto optimal for Pareto optimization problem (Gao & Yuan, 2015). The following lemma deduced from Theorem 7 in Hoogeveen (2005) shows the relationship between our initial problem and the corresponding constrained optimization problems.

Lemma 1. *Let x be the optimal objective value of problem $\alpha|\beta, C_{max}^B \leq \epsilon|C_{max}^A$, and let y be the optimal objective value of problem $\alpha|\beta, C_{max}^A \leq \epsilon|C_{max}^B$, then (x, y) is a Pareto optimal point of problem $\alpha|\beta|P(C_{max}^A, C_{max}^B)$.*

Lemma 1 implies that if there is an optimal algorithm for the constrained optimization problem $\alpha|\beta, C_{max}^B \leq \epsilon|C_{max}^A$, all the Pareto optimal

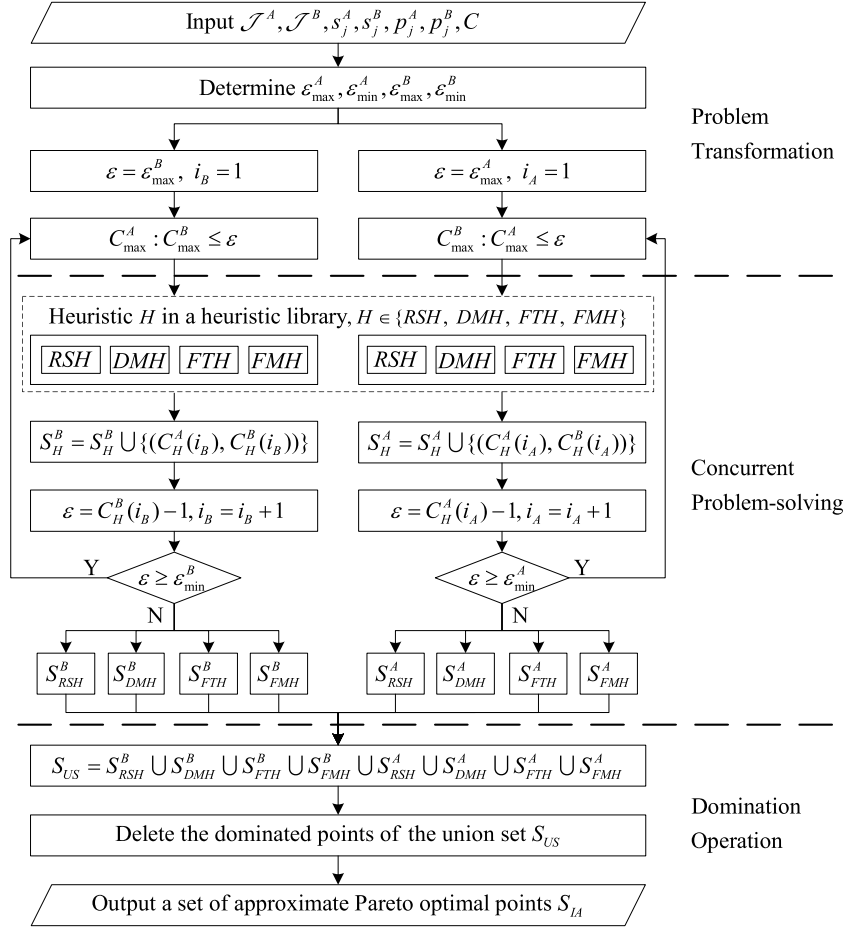


Fig. 2. Flow chart of the integrated algorithm.

points can be obtained using the ϵ -constraint method. Thus, the Pareto front induced by problem $\alpha|\beta, C_{\max}^B \leq \epsilon|C_{\max}^A$ is the same as that induced by problem $\alpha|\beta, C_{\max}^A \leq \epsilon|C_{\max}^B$.

However, these two constrained optimization problems $C_{\max}^A : C_{\max}^B \leq \epsilon$ and $C_{\max}^B : C_{\max}^A \leq \epsilon$ are both strongly NP -hard (Wang et al., 2017). It is impossible to obtain an optimal schedule of either problem in polynomial time, unless $P = NP$. Under heuristics, the Pareto fronts obtained for these two constrained optimization problems may be different, as demonstrated in our later experiments. When determining which objective should be used as the primary objective during problem transformation, both $C_{\max}^A$ and $C_{\max}^B$ are considered as the primary objectives. That is, both $C_{\max}^A : C_{\max}^B \leq \epsilon$ and $C_{\max}^B : C_{\max}^A \leq \epsilon$ will be solved in the second component of our algorithm.

5.2. Component (2): concurrent problem-solving

The concurrent problem-solving component contains a standard operating procedure where all heuristics used are stored in a heuristic library and loaded through a fixed algorithm interface. Essentially, any new efficient heuristic can be added into the heuristic library. Also, any heuristic stored in the heuristic library can be updated to the latest version. The concurrent problem-solving component delivers the approximate Pareto optimal points found by any heuristic in the heuristic library.

Next, we illustrate how to generate a set of approximate Pareto optimal points for the original Pareto optimization problem using the ϵ -constraint method. Without loss of generality, we use $C_{\max}^A : C_{\max}^B \leq \epsilon$ as

an example. Let S_H^B be the set of approximate Pareto optimal points obtained by heuristic H when solving problem $C_{\max}^A : C_{\max}^B \leq \epsilon$. Here, we use $\epsilon_{\max}^A = \epsilon_{\max}^B = C_{FFD}^+$ according to Theorem 3, and use $\epsilon_{\min}^X = C_{FFD}^X$, $X \in \{A, B\}$. Here, we show the ϵ -constraint algorithm by iteratively solving each constrained optimization problem with different values of ϵ , which is shown as follows.

Algorithm 1: ϵ -constraint algorithm.

Input: Heuristic H , C_{FFD}^B , C_{FFD}^+ , C , ϵ , J^A , J^B , n_A , n_B , n , p_j^X , s_j^X , $j = 1, \dots, n_X$, $X \in \{A, B\}$

Output: set of approximate Pareto optimal points S_H^B and corresponding schedules

```

1 Initialization:  $S_H^B = \emptyset, i_B = 1$ ;
2  $\epsilon_{\max}^B = C_{FFD}^+, \epsilon_{\min}^B = C_{FFD}^B$ ;
3 Set  $\epsilon = \epsilon_{\max}^B$ ;
4 while  $\epsilon \geq \epsilon_{\min}^B$  do
5   Solve  $C_{\max}^A : C_{\max}^B \leq \epsilon$  using heuristic  $H$ , and obtain point
    $(C_H^A(i_B), C_H^B(i_B))$ ;
6    $S_H^B = S_H^B \cup \{(C_H^A(i_B), C_H^B(i_B))\}$ ;
7    $\epsilon = C_H^B(i_B) - 1$ ;
8    $i_B = i_B + 1$ ;
9 end

```

In the concurrent problem-solving component, the integrated algorithm can run multiple heuristics in parallel. Based on different

strategies, we introduce four heuristics, namely Reserved Space Heuristic (*RSH*) (Wang et al., 2015), Dynamic Mix Heuristic (*DMH*) (Wang et al., 2015), Feasibility Test Heuristic (*FTH*) and Feasibility Mix Heuristic (*FMH*). The idea of these four strategies is intuitive, *RSH* focuses on the interests of the agent with bounded objective, *DMH* focuses on the machine utilization, *FTH* focuses on the selection of jobs improving machine utilization, and *FMH* focuses on combination of jobs.

5.2.1. Heuristic *RSH*

We design a reserved space heuristic to solve constrained problems. Specifically, to satisfy the constraint $C_{\max}^B \leq \epsilon$, *RSH* reserves enough batches in advance for the J^B -jobs in the first ϵ batches, and then assigns as many of the J^A -jobs as possible to the remaining batches, which follows the idea of Wang et al. (2015). Assume that the last r batches are reserved for the J^B -jobs. Heuristic *RSH* first assigns as many J^A -jobs as possible to batches $B_1, \dots, B_{\epsilon-r}$. Then, *RSH* assigns the J^B -jobs to batches using the first fit decreasing (*FFD*) rule. Finally, assign the remaining J^A -jobs to batches using the *FFD* rule. The pseudocode of *RSH* is shown as follows.

Heuristic *RSH*: Reserved space heuristic.

Input: $C, \epsilon, n_A, n_B, m, n, p_j^X, s_j^X, j = 1, \dots, n_X, X \in \{A, B\}$
Output: A schedule, $C_{\max}^A$ and $C_{\max}^B$

- 1 Initialization: $B_i = \emptyset, i = 1, \dots, n; T_A = \emptyset;$
- 2 Calculate the makespan of J^B -jobs using the *FFD* rule, denoted as r , which is the number of reserved batches for J^B -jobs;
- 3 **for** $j = 1$ to n_A **do**
- 4 **for** $k = 1$ to $\epsilon - \lceil r/m \rceil$ **do**
- 5 **for** $i = 1$ to m **do**
- 6 **if** Batch B_{ik} has enough space for job J_j^A **then**
- 7 Assign job J_j^A to batch $B_{ik};$
- 8 $T_A = T_A \cup \{J_j^A\};$
- 9 Break;
- 10 **end**
- 11 **end**
- 12 **end**
- 13 **end**
- 14 Rearrange the batches containing J^A -jobs in a nonincreasing order of their remaining spaces;
- 15 Assign all the J^B -jobs to batches using the *FFD* rule;
- 16 Rearrange the $(\epsilon - \lceil r/m \rceil + 1)$ -th batch to ϵ -th batch on each machine in a nonincreasing order of their remaining spaces;
- 17 Assign J^A -jobs in $J^A \setminus T_A$ to batches using the *FFD* rule.

5.2.2. Heuristic *DMH*

We observe that mixing and scheduling all the J^A -jobs and J^B -jobs together may improve the machine utilization, and may result in a small makespan of J^A . In order to implement the mix strategy and satisfy the constraint $C_{\max}^B \leq \epsilon$ simultaneously, for each batch B_i ($1 \leq i \leq \epsilon$), *DMH* reserves a part of space in B_i for the J^B -jobs, mixes all the unassigned J^B -jobs and the unassigned J^A -jobs together, and then assigns the mixed jobs to batch B_i using the *FFD* rule, which follows the idea of Wang et al. (2015). To decide how much the reserved space should be set aside in each batch, a dynamic adjustment strategy is employed based on two parameters: N_{job} and N_{batch} . Here, N_{job} is the number of unassigned J^B -jobs, and its initial value is n_B . N_{batch} is the number of empty batches before ϵ . For each batch, heuristic *DMH* assigns as many J^B -jobs as possible to the batch as long as the number of J^B -jobs is not greater than $\lceil N_{\text{job}}/N_{\text{batch}} \rceil$. The pseudocode of heuristic *DMH* is shown as follows.

Heuristic *DMH*: Dynamic mix heuristic.

Input: $C, \epsilon, n_A, n_B, m, n, p_j^X, s_j^X, j = 1, \dots, n_X, X \in \{A, B\}$
Output: A schedule, $C_{\max}^A$ and $C_{\max}^B$

- 1 Initialization: $J^A = J^A, J^B = J^B, N_{\text{job}} = n_B, N_{\text{batch}} = m\epsilon;$
- 2 **while** $N_{\text{job}} > 0$ **do**
- 3 $i = \lceil (m\epsilon - N_{\text{batch}})/m \rceil;$
- 4 $k = \text{mode}(m\epsilon - N_{\text{batch}}, m);$
- 5 **for** $j = 1$ to n_B **do**
- 6 **if** $J_j^B \in J^B$ and $C - \sum_{J_k^A \in B_i} s_k^A - \sum_{J_l^B \in B_i} s_l^B \geq s_j^B$ and $|B_{ik}| < \lceil N_{\text{job}}/N_{\text{batch}} \rceil$ **then**
- 7 Assign job J_j^B to batch $B_{ik};$
- 8 $J^B = J^B \setminus \{J_j^B\};$
- 9 **end**
- 10 **end**
- 11 Assign the jobs from $J^A \cup J^B$ to batch B_{ik} using the *FFD* rule;
- 12 Update sets J^A and $J^B;$
- 13 $N_{\text{job}} = |J^B|, N_{\text{batch}} = N_{\text{batch}} - 1;$
- 14 **end**
- 15 Assign the J^A -jobs in set J^A to batches using the *FFD* rule;
- 16 Rearrange batches before ϵ to ensure that the batches containing J^A -jobs are placed before the batches only containing J^B -jobs;

5.2.3. Heuristic *FTH*

Different from the reservation strategies for *RSH* and *DMH*, we propose a new heuristic *FTH* based on a feasibility strategy. The idea of heuristic *FTH* is to choose and assign as many J^A -jobs as possible before ϵ . To satisfy the constraint $C_{\max}^B \leq \epsilon$, the selected J^A -jobs needs to pass the following feasibility test. We test J^A -jobs in a nonincreasing order of their sizes. For a to-be-tested J^A -job J_j^A , search along the sequence of batches $B_{11}, B_{12}, \dots, B_{m\epsilon}$. If batch B_{ik} satisfying the following two conditions:

- (1) batch B_{ik} has enough space for job J_j^A ,
- (2) after job J_j^A is assigned to batch B_{ik} , the J^B -jobs complete before ϵ ,

then job J_j^A passes the feasibility test and is assigned to batch B_{ik} ; otherwise, job J_j^A is added to set L_A . Finally, the J^A -jobs in L_A are assigned to batches using the *FFD* rule. We show the details of heuristic *FTH* as follows.

5.2.4. Heuristic *FMH*

Heuristic *FMH* combines the ideas of the heuristics *FTH* and *DMH*, and intends to mix as many J^A -jobs as possible with J^B -jobs following a series of feasibility tests. Heuristic *FMH* conducts feasibility test for J^A -jobs according to a nonincreasing order of their job sizes. For job J_j^A ($1 \leq j \leq n_A$) from J^A , heuristic *FMH* temporarily adds job J_j^A to the selected job set SS_A , unites SS_A with J^B to form a mixed job set, and tests whether the mixed job set can be scheduled before ϵ using the *FFD* rule. If the mixed job set can be scheduled before ϵ , then job J_j^A passes the feasibility test, and is added to the set SS_A ; otherwise job J_j^A is moved to the set L_A . Finally, heuristic *FMH* assigns the jobs in $SS_A \cup J^B$ to batches using the *FFD* rule, and then assigns J^A -jobs in set L_A to batches using the *FFD* rule. We show the details of heuristic *FMH* as follows.

5.2.5. Relationship of the four heuristics

Heuristics *RSH*, *DMH*, *FTH* and *FMH* employ different strategies to satisfy the hard constraint. Without loss of generality, we show the details for the constrained optimization problem $C_{\max}^A : C_{\max}^B \leq \epsilon$ as follows.

Heuristic FTH: Feasibility test heuristic.

Input: $C, \epsilon, n_A, n_B, m, n, p_j^X, s_j^X, j = 1, \dots, n_X, X \in \{A, B\}$
Output: A schedule, $C_{\max}^A$ and $C_{\max}^B$

```

1 Initialization:  $T^A = \emptyset, L_A = \emptyset$ ;
2 for  $j = 1$  to  $n_A$  do
3   for  $k = 1$  to  $\epsilon$  do
4     for  $i = 1$  to  $m$  do
5       if Batch  $B_{ik}$  has enough space for job  $J_j^A$  then
6         Assign job  $J_j^A$  to batch  $B_{ik}$ ;
7         Calculate the makespan  $C_{\max}^B$  of  $J^B$ -jobs using
           the FFD rule;
8         if  $C_{\max}^B \leq \epsilon$  then
9            $T^A = T^A \cup \{J_j^A\}$ ;
10          Rearrange the obtained batches of each
             machine in a nondecreasing order of
             their remaining spaces;
11          Break;
12        else
13          Delete job  $J_j^A$  from batch  $B_{ik}$ ;
14        end
15      end
16    end
17  end
18 end
19  $L_A = J^A \setminus T^A$ ;
20 Assign all  $J^B$ -jobs to batches using the FFD rule;
21 Assign the jobs in set  $L_A$  to batches using the FFD rule.
```

Heuristic FMH: Feasibility mix heuristic.

Input: $C, \epsilon, n_A, n_B, m, n, p_j^X, s_j^X, j = 1, \dots, n_X, X \in \{A, B\}$
Output: A schedule, $C_{\max}^A$ and $C_{\max}^B$

```

1 Initialization:  $SS_A = \emptyset, L_A = \emptyset$ ;
2 for  $j = 1$  to  $n_A$  do
3   Calculate the makespan  $C_{\max}$  of all jobs in
      $SS_A \cup \{J_j^A\} \cup J^B$  using the FFD rule;
4   if  $[C_{\max}/m] \leq \epsilon$  then
5      $SS_A = SS_A \cup \{J_j^A\}$ ;
6   else
7      $L_A = L_A \cup \{J_j^A\}$ ;
8   end
9   Assign all the jobs in  $SS_A \cup J^B$  to batches using the FFD
     rule;
10  Assign all the jobs in  $L_A$  to batches using the FFD rule;
11  Rearrange batches  $B_1, \dots, B_\epsilon$  to ensure that the batches
     containing  $J^A$ -jobs are placed before the batches only
     filled with  $J^B$ -jobs.
12 end
```

- Both heuristics *RSH* and *DMH* use a reservation strategy, which prefers the J^B -jobs and reserves enough space in advance for J^B -jobs without considering J^A -jobs. Specifically, both heuristics *RSH* and *DMH* first ensure the assignment of all J^B -jobs and then schedule as many J^A -jobs as possible before time ϵ . Moreover, the major difference between *RSH* and *DMH* lies in the reservation way in the batches before ϵ . Specifically, heuristic *RSH* reserves a number of whole batches before time ϵ , while heuristic *DMH* reserves a part of space of each batch before time ϵ .
- Both heuristics *FTH* and *FMH* use a feasibility strategy, which prefers J^A -jobs and tests the feasibility of assigning a J^A -job under the condition of ensuring all J^B -jobs assigned. Specifically, both

FTH and *FMH* prefer to select as many J^A -jobs as possible, and further assign the selected J^A -jobs and all J^B -jobs before time ϵ . Moreover, the major difference between *FTH* and *FMH* lies in the way how feasibility test is conducted and how the selected jobs from J^A are assigned after passing the feasibility test. Specifically, heuristic *FTH* assigns a to-be-tested J^A -job to a fit batch before the feasibility test, and once it passes the feasibility test, the job is fixed in the fit batch. Whereas, heuristic *FMH* mixes a to-be-tested J^A -job and all the J^B -jobs together before the feasibility test, and once it passes the feasibility test, the job is added to a selected job set and assigned later after all the selected J^A -jobs are determined.

Since the given heuristics search in region R_{approx} , our algorithm is 2-approximate Pareto optimal.

With the advantages of unified framework, standard solving flow and fixed algorithm interface, the proposed integrated algorithm allows us to encapsulate the heuristics with the strategies of reserving space for J^B -jobs, and conducting feasibility test for J^A -jobs. Moreover, the encapsulated heuristics can be updated using new rules to replace *FFD* rule used in our heuristics, such as *FFI* (First Fit Increasing) rule, *BFD* (Best Fit Decreasing) rule, and *FFD_a* rule (Boyar & Favrholt, 2010). All the heuristics can be updated, encapsulated and added, and further complement and reinforce each other. Consequently, our integrated algorithm is more powerful than a single heuristic to generate an approximate Pareto optimal set with high quality.

6. Performance comparison and analysis

6.1. Experimental design

To generate representative and random instances, we vary in our experiments the following four factors: machine capacity C , sizes of the jobs, total number of jobs n , and number of J^A -jobs n_A . We set the machine capacity to be 100, i.e., $C = 100$, for all instances. The sizes of all jobs are generated using a uniform discrete distribution, i.e., $s_j^X \sim UI[1, 100]$. We vary the total number of jobs n from 10 to 100 with steps of 10, denoted as $n = 10, 20, \dots, 100$. For the ratio of n_A and n , we set $n_A/n = 0.2, 0.4, 0.6, 0.8$. According to the parameters n_A and n , there are $10 \times 4 = 40$ testing cases in total. For each testing case, 50 instances are randomly generated. Thus, we generate $40 \times 50 = 2,000$ instances in total. The proposed integrated algorithm is coded in MATLAB 9.5 and run on Intel(R) Core(TM) i5-7500 CPU with 8.00GB RAM under Windows 10 environment.

6.2. Performance comparison of algorithms

We first compare the performances of heuristics *RSH*, *DMH*, *FTH* and *FMH* based on coverage metric and contribution ratio. Then, we evaluate the performance of the integrated algorithm by comparing it with the non-dominated sorting genetic algorithm (*NSGA-II*), which is a widely used algorithm to find Pareto optimal points (Deb et al., 2002).

In *NSGA-II*, we encode chromosomes based on job sizes. We adopt a linear order crossover with a crossover probability $p_c = 0.9$, a swap mutation with a mutation probability $p_m = 0.15$. We employ elitism strategy based on non-dominated sorting and crowded-comparison operator. The maximum number of evolution generations is set to 10 generations, and the population of size is set to 40.

6.2.1. Metrics for performance evaluation

To evaluate the performance of heuristics, two metrics, coverage metric and contribution ratio, are introduced in this section.

(1) The coverage metric $C(S_X, S_Y)$ of sets S_X and S_Y , which is defined as follows (Zitzler & Thiele, 1999):

$$C(S_X, S_Y) = \frac{|\{y \in S_Y | \exists x \in S_X : x \succeq y\}|}{|S_Y|},$$

where $x \succeq y$ means x dominates or equals to y .

Obviously, $C(S_X, S_Y) \in [0, 1]$. The metric $C(S_X, S_Y)$ is the percentage of points that S_Y is dominated by S_X . $C(S_X, S_Y)$ and $C(S_Y, S_X)$ are not the same, thus $C(S_X, S_Y)$ and $C(S_Y, S_X)$ are needed in the experiments. By comparing $C(S_X, S_Y)$ and $C(S_Y, S_X)$, we can obtain the performance of sets S_X and S_Y . If $C(S_X, S_Y) > C(S_Y, S_X)$, then the points that S_X dominates S_Y are more than that S_Y dominates S_X , i.e., S_X outperforms S_Y .

The coverage metric compares a heuristic with the other heuristics, and computes the percentage of points dominated by the other heuristic. It conducts a relative analysis based on pairwise comparison, and takes the other heuristic as a reference to locally evaluate the performance of a heuristic.

(2) For an instance, the contribution ratio of heuristic H is defined as

$$CR_H = \frac{|S_H \cap S_{IA}|}{|S_{IA}|},$$

where S_H and S_{IA} are the approximate Pareto optimal set obtained by heuristic H and our integrated algorithm IA , respectively.

The contribution ratio $CR_H \in [0, 1]$. The bigger the contribution ratio of H is, the better heuristic H is. When $CR_X > CR_Y$, the number of points in set S_X accepted by the integrated algorithm is greater than that in set S_Y accepted by the integrated algorithm. The inequation $CR_X > CR_Y$ means that heuristic X outperforms heuristic Y .

The contribution ratio evaluates the performance of heuristics in different ways from the coverage metric. This ratio compares a heuristic with the integrated algorithm, and computes the percentage of points that the heuristic contributes to the integrated algorithm. Thus, this ratio takes the final set S_{IA} as a reference to globally evaluate the performance of a heuristic.

6.2.2. Results of performance comparison

The results of coverage metric is shown in Table 3. Each data entry in Table 3 is the mean value of coverage metrics for 50 instances.

Let S_H be the set of approximate Pareto optimal points obtained by heuristic H , where $H \in \{RSH, DMH, FTH, FMH\}$. Set S_H is obtained by deleting the dominated points in $S_H^A \cup S_H^B$, where S_H^A and S_H^B are the sets of approximate Pareto optimal points obtained by heuristic H for problems $C_{\max}^A : C_{\max}^A \leq \epsilon$ and $C_{\max}^B : C_{\max}^B \leq \epsilon$, respectively. Let S_{US} be the union set of $S_{RSH}, S_{DMH}, S_{FTH}$ and S_{FMH} . Let S_{IA} be the set of approximate Pareto optimal points of set S_{US} , and $S_{NSGA-II}$ be the set of approximate Pareto optimal points obtained by $NSGA-II$.

Altogether, we have fourteen coverage metrics as indicators of relative performance: $C(S_{RSH}, S_{FTH}), C(S_{FTH}, S_{RSH}), C(S_{RSH}, S_{DMH}), C(S_{DMH}, S_{RSH}), C(S_{RSH}, S_{FMH}), C(S_{FMH}, S_{RSH}), C(S_{FTH}, S_{DMH}), C(S_{DMH}, S_{FTH}), C(S_{FTH}, S_{FMH}), C(S_{FMH}, S_{FTH}), C(S_{DMH}, S_{FMH}), C(S_{FMH}, S_{DMH}), C(S_{IA}, S_{NSGA-II})$ and $C(S_{NSGA-II}, S_{IA})$, short for $C_{RS-FM}, C_{FT-RS}, C_{RS-DM}, C_{DM-RS}, C_{RS-FM}, C_{FM-RS}, C_{FT-DM}, C_{DM-FT}, C_{FT-FM}, C_{FM-FT}, C_{DM-FM}, C_{FM-DM}, C_{IA-N}$ and C_{N-IA} , respectively.

Table 3 shows that the performance of four heuristics is ranked as $FTH > RSH > DMH > FMH$, which are based on the following facts.

- From the last row of Table 3, we compare the average coverage metrics of any two heuristics and obtain Table 4.
- We also count the number of cases that a heuristic outperforms another heuristic. Specifically, in Table 5, each entry shows the number of instances for which the heuristic in the corresponding row outperforms the heuristic in the corresponding column. Since there are a few cases with the equal value for different heuristics, it is omitted in Table 5. For example, RSH outperforms DMH on 60% of cases, DMH outperforms RSH on 37.5% of cases, and one case with the same performance is omitted. Thus, RSH outperforms DMH .

We further compare the integrated algorithm with $NSGA-II$. An immediate observation from columns 15 and 16 in Table 3 is that $C(S_{IA}, S_{NSGA-II}) = 1$ for almost all testing cases, and

$C(S_{NSGA-II}, S_{IA}) = 0$ for most testing cases. This means that the approximate Pareto optimal points obtained by the integrated algorithm dominate that obtained by $NSGA-II$ for most testing cases.

We compute the average contribution ratios of different combinations of heuristics for 2000 instances, which is shown in Tables 6, 7 and 8, respectively. In Table 6, $CR_{FT}, CR_{RS}, CR_{DM}$ and CR_{FM} denote the contribution ratios of heuristics FTH, RSH, DMH and FMH , respectively. In Table 7, $CR_{FT-RS}, CR_{FT-DM}, CR_{FT-FM}, CR_{RS-DM}, CR_{RS-FM}$ and CR_{DM-FM} denote the contribution ratios of two combined heuristics $FTH \& RSH, FTH \& DMH, FTH \& FMH, RSH \& DMH, RSH \& FMH$ and $DMH \& FMH$, respectively. In Table 8, $CR_{FT-RS-DM}, CR_{FT-RS-FM}, CR_{FT-DM-FM}$ and $CR_{RS-DM-FM}$ denote the contribution ratios of three combined heuristics $FTH \& RSH \& DMH, FTH \& RSH \& FMH, FTH \& DMH \& FMH$ and $RSH \& DMH \& FMH$, respectively.

Table 6 also shows that the performance of heuristics can be ranked as $FTH > RSH > DMH > FMH$. Tables 7 and 8 show that, for heuristics FTH, RSH and DMH , the better the integrated algorithm adopt the heuristic, the better the obtained approximate Pareto optimal set is. For example, in Table 7, combining heuristics FTH and RSH together contributes 97.49% to the integrated algorithm, and combining heuristics FTH and DMH together contributes 96.94% to the integrated algorithm. For heuristic FMH , though it is the worst among the four heuristics, it is not entirely useless for our integrated algorithm. Instead, good results are obtained by combining heuristic FMH into different combinations of heuristics. Specifically, in Table 7, combining heuristics RSH and DMH together contributes 94.74% to the integrated algorithm, while combining heuristics RSH and FMH together contributes 96.24% to the integrated algorithm. In Table 8, combining heuristics FTH, RSH and DMH together contributes 98.79% to the integrated algorithm, while combining heuristics FTH, RSH and FMH together contributes 99.04% to the integrated algorithm.

We provide the results of running times of the testing cases of $n_A/n = 0.2$ and $n = 10, 50, 100$ in Table 9, which shows that the integrated algorithm runs much faster than $NSGA-II$. Thus, the integrated algorithm is advantageous over $NSGA-II$ in terms of both solution quality and running time.

6.3. Quality evaluation of S_{IA}

In this section, we further examine the quality of the set of approximate Pareto optimal points S_{IA} , and analyze the closeness of S_{IA} to the Pareto front using the following two method. The first method is based on the distance metrics D_{avg} and D_{max} defined in Section 4.3. Note that the smaller the values of D_{avg} and D_{max} are, the nearer S_{IA} is to S_{RS} , and the better the integrated algorithm performs. The second method is based on the lower bound set and the reference set. As shown in Theorem 6, if $S_{IA} = S_{RS}$, then S_{IA} is the Pareto front. We count the number of instances that $S_{IA} = S_{RS}$, denoted by I_{RS} , and the number of instances that $S_{IA} = S_{LB}$, denoted by I_{LB} . Note that $I_{LB} \geq I_{RS}$, since $S_{RS} \subseteq S_{LB}$. The larger the value of I_{RS} and I_{LB} are, the better the integrated algorithm performs.

6.3.1. Results of D_{avg} and D_{max}

The computational results of D_{avg} and D_{max} are shown in Figs. 3 and 4, respectively. In each figure, there are four subgraphs representing the cases of $n_A/n = 0.2, n_A/n = 0.4, n_A/n = 0.6$ and $n_A/n = 0.8$, respectively. Furthermore, in each subgraph, the x-axis is the total number of jobs, varying from 10 to 100, and the y-axis is the value of D_{avg} in Fig. 3 (respectively, D_{max} in Fig. 4). Each data point in the graph is the average value of 50 instances, and the maximum value and minimum value are also illustrated by error bars.

From Figs. 3 and 4, we can see that the average values of both D_{avg} and D_{max} are very small (less than 0.1), which means the set of approximate Pareto optimal points S_{IA} is very close to the Pareto front for

Table 3
Performance comparison results.

n	n_A/n	C_{KS-FT}	C_{FT-RS}	C_{RS-DM}	C_{DM-RS}	C_{RS-FM}	C_{FM-RS}	C_{FT-DM}	C_{DM-FT}	C_{FT-FM}	C_{FM-FT}	C_{DM-FM}	C_{FM-DM}	C_{IA-N}	C_{N-IA}
10	0.2	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.8000
10	0.4	0.9933	1.0000	0.9933	1.0000	0.9933	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.7200
10	0.6	0.9933	0.9900	0.9933	0.9900	0.9933	0.9900	0.9933	1.0000	0.9933	1.0000	1.0000	1.0000	0.9900	0.7200
10	0.8	1.0000	0.9933	1.0000	0.9933	1.0000	0.9933	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9800	0.8200
20	0.2	0.9733	0.9867	0.9867	0.9833	0.9667	0.9667	0.9933	0.9700	0.9867	0.9733	0.9667	0.9767	1.0000	0.3833
20	0.4	0.9767	1.0000	0.9667	1.0000	0.9633	0.9600	0.9850	0.9950	0.9850	0.9550	0.9850	0.9483	1.0000	0.2300
20	0.6	0.9633	1.0000	0.9733	0.9933	0.9500	0.9700	0.9850	0.9667	0.9867	0.9700	0.9533	0.9717	1.0000	0.2350
20	0.8	0.9833	1.0000	0.9800	1.0000	0.9700	0.9767	0.9933	0.9900	0.9867	0.9767	0.9833	0.9767	1.0000	0.4000
30	0.2	0.9550	1.0000	0.9833	0.9767	0.9517	0.9533	0.9900	0.9383	0.9900	0.9550	0.9517	0.9600	1.0000	0.0167
30	0.4	0.9800	0.9767	0.9717	0.9600	0.9667	0.8850	0.9750	0.9633	0.9867	0.8900	0.9633	0.8800	1.0000	0.0000
30	0.6	0.9300	0.9767	0.9403	0.9300	0.9217	0.9317	0.9637	0.9217	0.9583	0.9300	0.9233	0.9427	1.0000	0.0000
30	0.8	0.9900	0.9800	1.0000	0.9700	0.9900	0.9533	0.9933	0.9800	0.9933	0.9733	0.9733	0.9733	1.0000	0.0500
40	0.2	0.9533	0.9800	0.9633	0.9667	0.9433	0.8933	0.9767	0.9450	0.9733	0.8883	0.9683	0.9033	1.0000	0.0000
40	0.4	0.9433	0.9800	0.9437	0.9533	0.9603	0.8783	0.9693	0.9417	1.0000	0.8517	0.9593	0.8403	1.0000	0.0000
40	0.6	0.9727	0.9750	0.9593	0.9400	0.9617	0.9333	0.9703	0.9477	0.9783	0.9393	0.9550	0.9217	1.0000	0.0000
40	0.8	0.9550	0.9900	0.9700	0.9600	0.9517	0.9250	0.9867	0.9933	0.9833	0.9117	0.9567	0.9217	1.0000	0.0000
50	0.2	0.9300	0.9950	0.9350	0.9550	0.9233	0.9283	0.9650	0.9383	0.9750	0.8783	0.9550	0.8933	1.0000	0.0000
50	0.4	0.8933	0.9833	0.9217	0.9667	0.8933	0.8933	0.9617	0.9000	0.9483	0.8617	0.9350	0.8650	1.0000	0.0000
50	0.6	0.9500	0.9700	0.9483	0.9633	0.9333	0.8900	0.9600	0.9500	0.9617	0.8617	0.9550	0.8633	1.0000	0.0000
50	0.8	0.9417	0.9867	0.9467	0.9683	0.9450	0.9067	0.9933	0.9550	0.9867	0.8767	0.9700	0.8967	1.0000	0.0000
60	0.2	0.9300	0.9817	0.9560	0.9383	0.9300	0.8867	0.9767	0.9167	0.9817	0.8817	0.9450	0.8710	1.0000	0.0000
60	0.4	0.9370	0.9583	0.9413	0.9243	0.9140	0.8727	0.9453	0.9270	0.9660	0.8430	0.9430	0.8413	1.0000	0.0000
60	0.6	0.8883	0.9867	0.9193	0.9450	0.8800	0.8900	0.9833	0.8667	0.9783	0.8250	0.9267	0.8420	1.0000	0.0000
60	0.8	0.9383	0.9667	0.9383	0.9167	0.9267	0.8950	0.9717	0.8950	0.9683	0.8850	0.9000	0.8900	1.0000	0.0000
70	0.2	0.9243	0.9633	0.9427	0.9350	0.9283	0.8427	0.9883	0.9233	1.0000	0.8203	0.9600	0.8187	1.0000	0.0000
70	0.4	0.8933	0.9433	0.9253	0.8950	0.8783	0.8483	0.9467	0.8583	0.9533	0.8117	0.8850	0.8260	1.0000	0.0000
70	0.6	0.9090	0.9800	0.9167	0.8867	0.8803	0.8263	0.9567	0.8523	0.9570	0.7840	0.9170	0.7913	1.0000	0.0000
70	0.8	0.9370	0.9883	0.9430	0.9267	0.9320	0.8883	0.9717	0.9010	0.9850	0.8763	0.9343	0.8617	1.0000	0.0000
80	0.2	0.9577	0.9817	0.9810	0.9500	0.9527	0.9183	0.9833	0.9320	0.9900	0.9197	0.9433	0.9080	1.0000	0.0000
80	0.4	0.8930	0.9750	0.9210	0.9010	0.8917	0.8570	0.9543	0.8523	0.9733	0.8207	0.9067	0.8220	1.0000	0.0000
80	0.6	0.8647	0.9763	0.9037	0.9347	0.8483	0.8033	0.9503	0.8720	0.9683	0.7377	0.9100	0.7437	1.0000	0.0000
80	0.8	0.8617	0.9733	0.8837	0.9483	0.8450	0.8867	0.9217	0.8967	0.9550	0.8600	0.9117	0.8277	1.0000	0.0000
90	0.2	0.9193	0.9503	0.9400	0.8993	0.8930	0.8473	0.9533	0.8830	0.9453	0.8210	0.9250	0.8130	1.0000	0.0000
90	0.4	0.8925	0.9563	0.9158	0.9103	0.8750	0.8522	0.9693	0.8670	0.9650	0.7894	0.9250	0.7927	1.0000	0.0000
90	0.6	0.8417	0.9333	0.8927	0.9100	0.8317	0.8577	0.9167	0.8533	0.9600	0.8200	0.8767	0.7937	1.0000	0.0000
90	0.8	0.8883	0.9767	0.9337	0.9327	0.8867	0.9027	0.9627	0.8583	0.9683	0.8817	0.8800	0.8770	1.0000	0.0000
100	0.2	0.9183	0.9217	0.9227	0.9150	0.9083	0.8567	0.9483	0.9003	0.9583	0.8543	0.9293	0.8620	1.0000	0.0000
100	0.4	0.9043	0.9883	0.9073	0.8637	0.8810	0.8720	0.9693	0.8330	0.9550	0.8147	0.8910	0.8150	1.0000	0.0000
100	0.6	0.8720	0.9750	0.8860	0.9310	0.8567	0.8383	0.9310	0.9120	0.9333	0.8147	0.9150	0.7957	1.0000	0.0000
100	0.8	0.9010	0.9617	0.9313	0.9070	0.8993	0.8253	0.9667	0.8820	0.9833	0.7813	0.9277	0.8033	1.0000	0.0000
Average Value		0.9337	0.9775	0.9470	0.9460	0.9254	0.9024	0.9702	0.9232	0.9755	0.8834	0.9427	0.8833	0.9992	0.1094

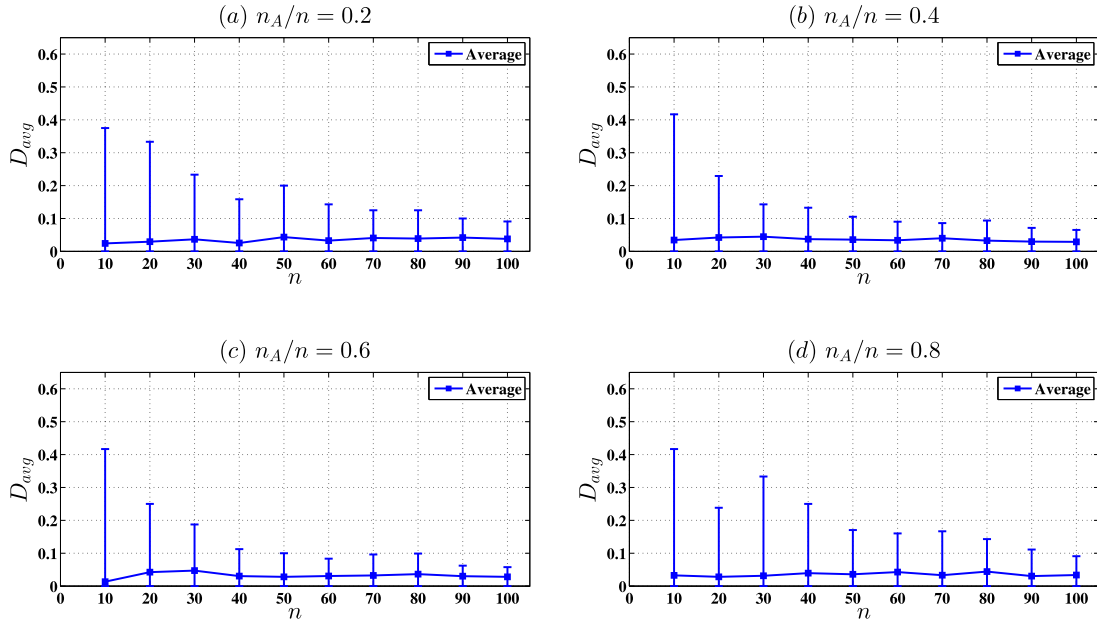
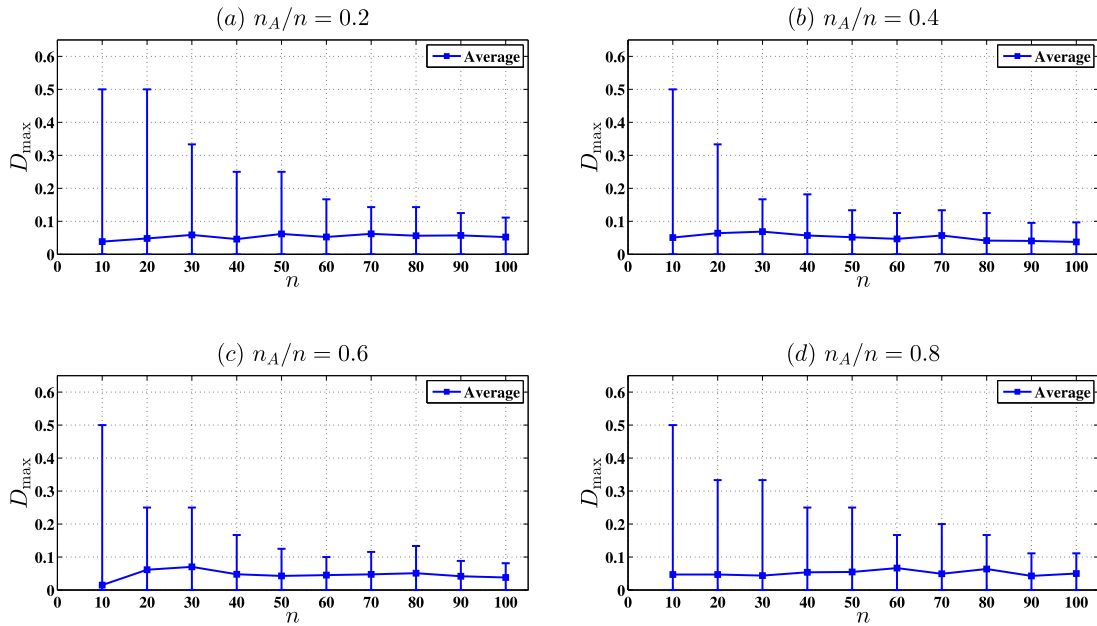
Fig. 3. Computational results of D_{avg} .Fig. 4. Computational results of D_{max} .

Table 4

Relationship of the average coverage metrics.

Average coverage metric of two heuristics	Meaning
$C(S_{FTH}, S_{RSH}) > C(S_{RSH}, S_{FTH})$	$FTH > RSH$
$C(S_{FTH}, S_{DMH}) > C(S_{DMH}, S_{FTH})$	$FTH > DMH$
$C(S_{FTH}, S_{FMH}) > C(S_{FMH}, S_{FTH})$	$FTH > FMH$
$C(S_{RSH}, S_{DMH}) > C(S_{DMH}, S_{RSH})$	$RSH > DMH$
$C(S_{RSH}, S_{FMH}) > C(S_{FMH}, S_{RSH})$	$RSH > FMH$
$C(S_{DMH}, S_{FMH}) > C(S_{FMH}, S_{DMH})$	$DMH > FMH$

Table 5

Number of cases that a heuristic outperforms another one.

Heuristic		FTH	RSH	DMH	FMH
FTH	>	–	35	35	36
RSH	>	4	–	24	28
DMH	>	2	15	–	32
FMH	>	1	10	4	–

most instances. In addition, the average values of D_{avg} and D_{max} do not change greatly with the number of jobs. Therefore, the integrated algorithm remains stable in generating the set of approximate Pareto optimal points S_{IA} with high quality.

6.3.2. Results of I_{LB} and I_{RS}

The results of I_{LB} and I_{RS} are illustrated in Fig. 5. The four subgraphs represent the case of $n_A/n = 0.2$, $n_A/n = 0.4$, $n_A/n = 0.6$ and $n_A/n = 0.8$, respectively. In each subgraph, the x-axis is the total number

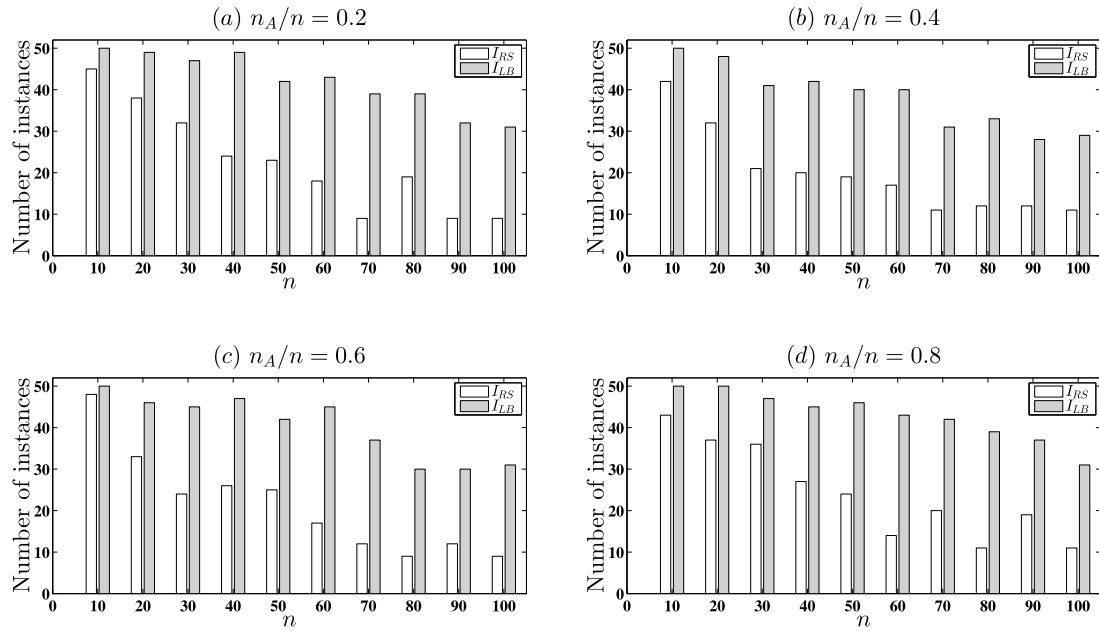
Fig. 5. Results of I_{LB} and I_{RS} .

Table 6

Average contribution ratios of one heuristic.

	CR_{FT}	CR_{RS}	CR_{DM}	CR_{FM}
Minimum	0	0	0	0
Maximum	1	1	1	1
Average	0.9533	0.9232	0.8974	0.8665
Variance	0.0170	0.0295	0.0342	0.0367

Table 7

Average contribution ratios of two heuristics.

	CR_{FT-RS}	CR_{FT-DM}	CR_{FT-FM}	CR_{RS-DM}	CR_{RS-FM}	CR_{DM-FM}
Minimum	0.3333	0	0	0	0.3333	0
Maximum	1	1	1	1	1	1
Average	0.9749	0.9694	0.9694	0.9474	0.9624	0.9469
Variance	0.0098	0.0110	0.0102	0.0197	0.0118	0.0161

Table 8

Average contribution ratios of three heuristics.

	$CR_{FT-RS-DM}$	$CR_{FT-RS-FM}$	$CR_{FT-DM-FM}$	$CR_{RS-DM-FM}$
Minimum	0.3333	0.5000	0	0.3333
Maximum	1	1	1	1
Average	0.9879	0.9904	0.9821	0.9805
Variance	0.0048	0.0029	0.0063	0.0059

Table 9

Average running times (seconds).

	RSH	FTH	DMH	FMH	IA	$NSGA-II$
$n = 10$	0.0186	0.0160	0.0039	0.0093	0.0639	3.9399
$n = 50$	0.3588	0.4428	0.0474	0.2171	1.0662	23.0050
$n = 100$	3.2547	3.3776	0.2498	1.6125	8.4947	71.1266

of jobs, varying from 10 to 100, and the y-axis is the number of instances. The grey bars and the unfilled bars represent I_{LB} and I_{RS} , respectively.

As shown in Fig. 5, there are a large number of instances for which the set of approximate Pareto optimal points S_{IA} belongs to the lower bound set. Take the case of $n_A/n = 0.8$ for example, the maximum of I_{LB} is 50 among 50 instances in the case of $n = 10, 20$, and the minimum of I_{LB} is 31 among 50 instances in the case of $n = 100$. Furthermore, there

are a number of instances for which the integrated algorithm obtains the reference set. Taking the case of $n_A/n = 0.6$ for example, the maximum of I_{RS} is 48 among 50 instances in the case of $n = 10$, and the minimum of I_{RS} is 9 among 50 instances in the case of $n = 80, 100$.

We further count the total values of I_{LB} and I_{RS} among the 2,000 instances and find that, for more than 1,630 (i.e., 81.5 %) instances, the approximate Pareto optimal points in S_{IA} are in the lower bound set, and for at least 880 (i.e., 44 %) instances, the integrated algorithm obtains the Pareto front. In addition, both I_{LB} and I_{RS} slightly decrease and gradually level off as the number of jobs increases regardless of the ratio of n_A/n .

Based on the above results, we conclude that the obtained set of approximate Pareto optimal points S_{IA} is very close to the Pareto front, and even is the Pareto front for a large number of instances.

7. Conclusions and future research

We study the Pareto optimization problem of scheduling jobs of two competing agents on a single parallel batch machine with equal processing times and non-identical job sizes. The objective is to find all the Pareto optimal points minimizing the makespans for both agents.

Theoretically, we analyze properties of the Pareto front, recognize an approximate Pareto region that covers the Pareto front and is obtainable in polynomial time, find an upper bound on the total number of Pareto optimal points, and prove that any obtained point in the approximate Pareto region is 2-approximate Pareto optimal. Then, we identify a lower bound set and a reference set to evaluate the quality of an obtained set of approximate Pareto optimal points. Furthermore, two effective performance metrics are introduced to measure the average distance and maximum distance from an obtained set of approximate Pareto optimal points to the reference set. We also provide a sufficient condition under which an obtained set of approximate Pareto optimal points is the Pareto front.

Algorithmically, we propose an integrated algorithm under a unified framework and standard solving flow, which consists of three components. The algorithm first transfers the Pareto optimization problem into two constrained optimization problems, and then uses a standard operating procedure that encapsulates, updates and loads different heuristics flexibly through a fixed algorithm interface. Based on different solving strategies to satisfy constraints in constrained optimization

problems, we introduce four heuristics, including Reserved Space Heuristic (*RSH*), Feasibility Test Heuristic (*FTH*), Dynamic Mix Heuristic (*DMH*) and Feasibility Mix Heuristic (*FMH*), to solve the two constrained optimization problems.

Experimental results show that, as measured by a coverage metric and a contribution ratio, the performance of the proposed heuristics is ranked as $FTH > RSH > DMH > FMH$, and the proposed integrated algorithm outperforms the widely used non-dominated sorting genetic algorithm (*NSGA-II*). Furthermore, based on two distance metrics, we find that the set of approximate Pareto optimal points obtained by the integrated algorithm is very close to the Pareto front, and even reaches the Pareto front for 44 % of the 2000 instances tested. Our results also demonstrate that, with the increase of the number of jobs, the integrated algorithm remains stable in generating a set of approximate Pareto optimal points with high quality.

Different heuristics use different strategies, and show different problem-solving abilities. On one hand, though each heuristic owns a certain problem-solving ability, each heuristic cannot completely outperform the other three heuristics. For the same instance, different heuristics may return different solutions, and for each heuristic, it is possible to return a best solution for some instances, which is verified in this section. On the other hand, different combinations of heuristics result in different solutions, and the combination of heuristics with high (respectively, low) performance may not outperform that with low (respectively, high) performance. Therefore, the heuristics complement and reinforce each other.

For future research, it is interesting to study whether the analysis of Pareto optimal region and integrated algorithm can be extended to the problem with arbitrary job processing times. It is valuable to consider that machines have availability constraints. Further, instead of makespan, other objectives, such as the total completion time, deserve future investigation.

CRedit authorship contribution statement

Cui-Lin Zhang: Conceptualization, Methodology, Software, Data curation, Investigation, Writing – original draft; **Guo-Qiang Fan:** Methodology, Software, Formal analysis, Visualization, Supervision, Writing – review & editing.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

The work was partly supported by the [Fundamental Funds for the Central Universities](#) (Grant No. ZYTS25004).

References

Agnetis, A., Billaut, J.-C., Gawiejnowicz, S., Pacciarelli, D., & Soukhal, A. (2014). Multiagent scheduling-models and algorithms. Springer.

- Agnetis, A., Mirchandani, P. B., Pacciarelli, D., & Pacifici, A. (2004). Scheduling problems with two competing agents. *Operations Research*, 52, 229–242.
- Baker, K.-R., & Smith, J.-C. (2003). A multiple-criterion model for machine scheduling. *Journal Scheduling*, 6, 7–16.
- Boyar, J., & Favrholdt, L. M. (2010). Scheduling jobs on grid processors. *Algorithmica*, 57(4), 819–847.
- Czyżżak, P., & Jaskiewicz, A. (1998). Pareto simulated annealing - a metaheuristic technique for multiple objective combinatorial optimization. *Journal Multi-Criteria Decis. Anal.*, 7(1), 34–47.
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182–197.
- Dósa, G., Li, R., Han, X., & Tuza, Z. (2013). Tight absolute bound for first fit decreasing bin-packing: $FFD(L) \leq 11/9 OPT(L) + 6/9$. *Theoretical Computer Science*, 510, 13–61.
- Ehr Gott, M., Shao, L., & Schöbel, A. (2011). An approximation algorithm for convex multi-objective programming problems. *Journal of Global Optimization*, 50(3), 397–416.
- Fan, B. Q., Cheng, T. C. E., Li, S. S., & Feng, Q. (2013). Bounded parallel-batching scheduling with two competing agents. *Journal Scheduling*, 16(3), 261–271.
- Fan, G.-Q., Wang, J.-Q., & Liu, Z. (2023). Two-agent scheduling on mixed batch machines to minimise the total weighted makespan. *International Journal of Production Research*, 61(1), 238–257.
- Feng, Q., Yuan, J. J., Liu, H., & He, C. (2013). A note on two-agent scheduling on an unbounded parallel-batching machine with makespan and maximum lateness objectives. *Applied Mathematical Modelling*, 37(10–11), 7071–7076.
- Gao, Y., & Yuan, J.-J. (2015). Pareto minimizing total completion time and maximum cost with positional due indices. *Journal of the Operations Research Society of China*, 3(3), 381–387.
- Garey, M. R., & Johnson, D. S. (1979). Computers and intractability: A guide to the theory of NP-completeness. WH Freeman New York.
- Graham, R. L., Lawler, E. L., Lenstra, J. K., & Kan, A. H. G. R. (1979). Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics*, 5, 287–326.
- Haimes, Y. V. (1971). On a bicriterion formulation of the problems of integrated system identification and system optimization. *IEEE Transactions on Systems, Man, and Cybernetics*, 1(3), 296–297.
- He, C., Lin, Y., & Yuan, J. J. (2007). Bicriteria scheduling on a batching machine to minimize maximum lateness and makespan. *Theoretical Computer Science*, 381(1–3), 234–240.
- He, C., Wu, J., & Lin, H. (2022). Two-agent bounded parallel-batching scheduling for minimizing maximum cost and makespan. *Discrete Optimization*, 45, 100698.
- Hoogeveen, H. (2005). Multicriteria scheduling. *European Journal of Operational Research*, 167(3), 592–623.
- Johnson, D. S. (1973). Near optimal bin packing algorithms. Doctoral thesis Massachusetts Institute of Technology Cambridge.
- Li, K., Zhang, H., Chu, C., Jia, Z.-h., & Chen, J. (2022). A bi-objective evolutionary algorithm scheduled on uniform parallel batch processing machines. *Expert Systems with Applications*, 204, 117487.
- Li, S., & Yuan, J. J. (2012). Unbounded parallel-batching scheduling with two competitive agents. *Journal Scheduling*, 15(5), 629–640.
- Martello, S., & Toth, P. (1990). Lower bounds and reduction procedures for the bin packing problem. *Discrete Applied Mathematics*, 28(1), 59–70.
- Perez-Gonzalez, P., & Framinan, J. M. (2014). A common framework and taxonomy for multicriteria scheduling problems with interfering and competing jobs: Multi-agent scheduling problems. *European Journal of Operational Research*, 235(1), 1–16.
- Sabouni, M. T. Y., & Jolai, F. (2010). Optimal methods for batch processing problem with makespan and maximum lateness objectives. *Applied Mathematical Modelling*, 34(2), 314–324.
- Tan, Q., Chen, H.-P., Du, B., & Li, X.-L. (2011). Two-agent scheduling on a single batch processing machine with non-identical job sizes. In *Aimsec* (pp. 7431–7435). IEEE.
- Ulungu, E. L., Teghem, J., & Ost, C. (1998). Efficiency of interactive multiobjective simulated annealing through a case study. *The Journal of the Operational Research Society*, 49, 1044–1050.
- Wang, J.-Q., Chen, J., Zhang, Y., & Huang, G. Q. (2016). Schedule-based execution bottleneck identification in a job shop. *Computers Industrial Engineering*, 98, 308–322.
- Wang, J.-Q., Fan, G.-Q., Zhang, Y., Zhang, C.-W., & Leung, J. Y. T. (2017). Two-agent scheduling on a single parallel-batching machine with equal processing time and non-identical job sizes. *European Journal of Operational Research*, 258(2), 478–490.
- Wang, J.-Q., Zhang, C.-W., Fan, G.-Q., & Sun, S.-D. (2015). Heuristics for non-dominated sets of two-agent scheduling on a single parallel-batching machine. In *International conference on practical applications of agents and multi-agent systems* (pp. 55–68). Springer.
- Zhang, C.-L., Wang, J.-Q., & Zhang, C.-W. (2019). Two-agent scheduling on a single parallel-batching machine to minimize the weighted sum of the agents' makespans. *Journal of Ambient Intelligence and Humanized Computing*, 10(3), 999–1007.
- Zitzler, E., & Thiele, L. (1999). Multiobjective evolutionary algorithms: A comparative case study and the strength pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4), 257–271.